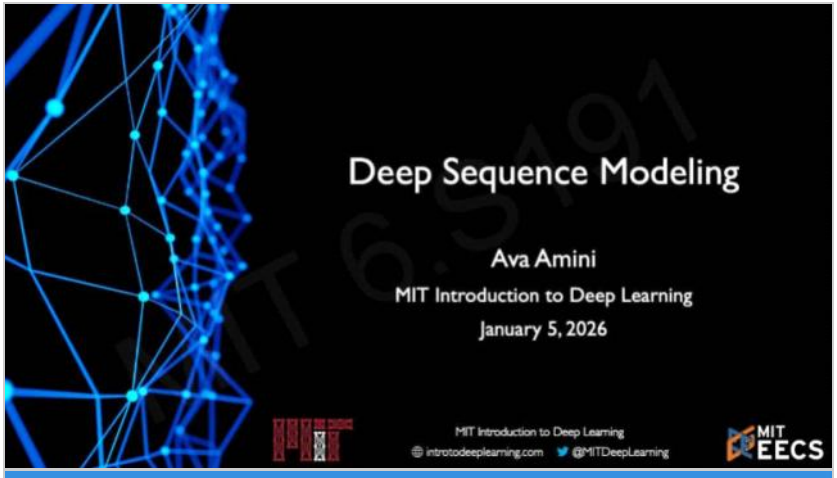
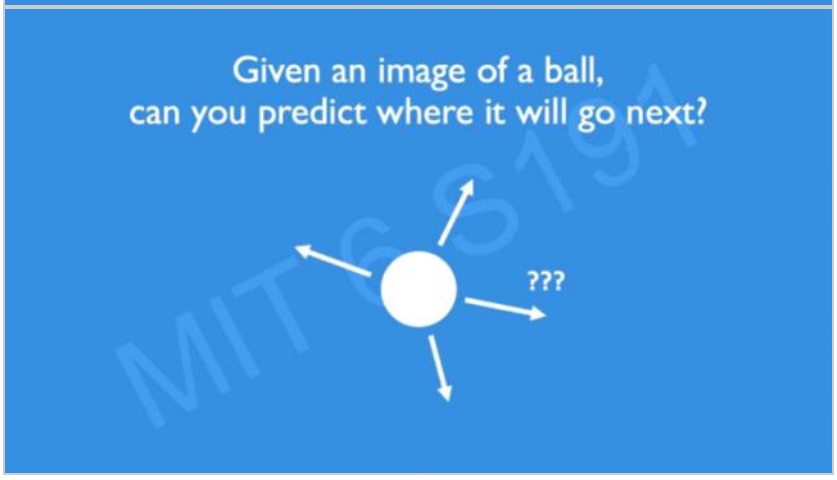
Lecture 2

Sunday, 12 April 2026 1:42 pm

6S191_MIT
_DeepLea...





Given an image of a ball,
can you predict where it will go next?

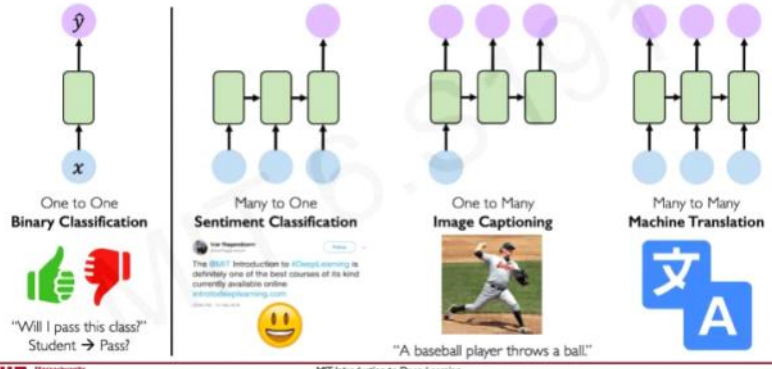
Given an image of a ball,
can you predict where it will go next?

Sequences in the Wild

Audio

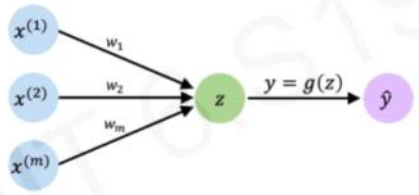
Sequences in the Wild

Sequence Modeling Applications

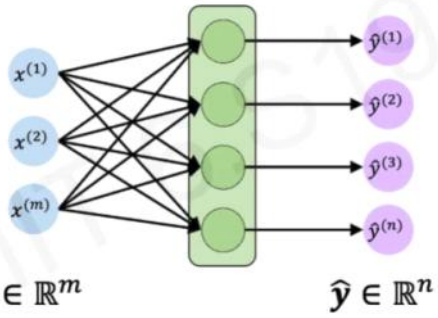


Neurons with Recurrence

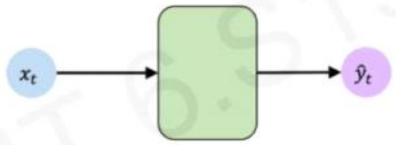
The Perceptron Revisited



Feed-Forward Networks Revisited

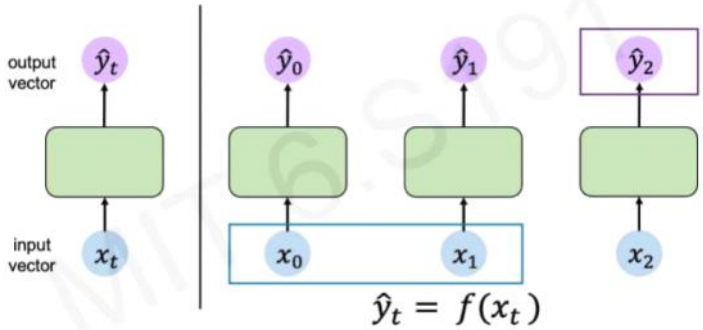


Feed-Forward Networks Revisited

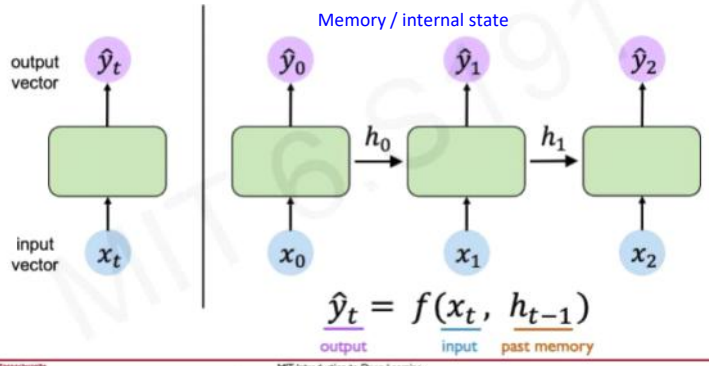


$\boldsymbol{x_t} \in \mathbb{R}^m$ $\hat{\boldsymbol{y}}_t \in \mathbb{R}^n$

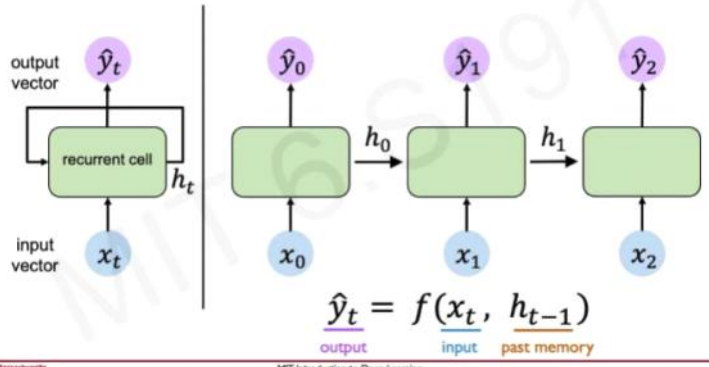
Handling Individual Time Steps



Neurons with Recurrence

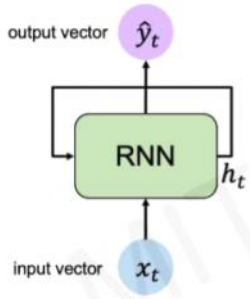


Neurons with Recurrence



Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs)



Apply a **recurrence relation** at every time step to process a sequence:

$$\boxed{h_t} = \boxed{f_W}(\boxed{x_t}, \boxed{h_{t-1}})$$

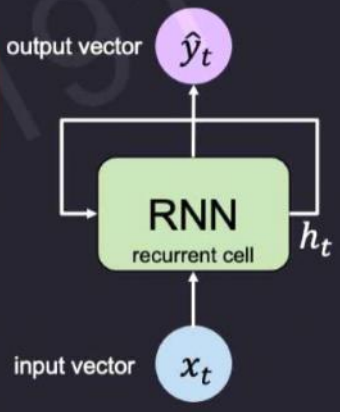
cell state — function with weights W — input — old state

Note: the same function and set of parameters are used at every time step

RNNs have a **state**, h_t, that is updated **at each time step** as a sequence is processed

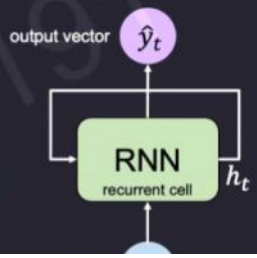
RNN Intuition

```
my_rnn = RNN()
hidden_state = [0, 0, 0, 0]

sentence = ["I", "love", "recurrent", "neural"]

for word in sentence:
    prediction, hidden_state = my_rnn(word, hidden_state)

next_word_prediction = prediction
# >>> "networks!"
```



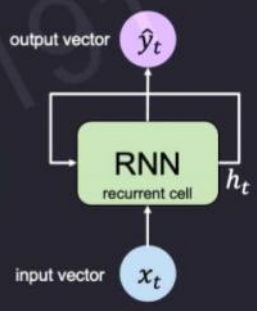
RNN Intuition

```
my_rnn = RNN()
hidden_state = [0, 0, 0, 0]

sentence = ["I", "love", "recurrent", "neural"]

for word in sentence:
    prediction, hidden_state = my_rnn(word, hidden_state)

next_word_prediction = prediction
```



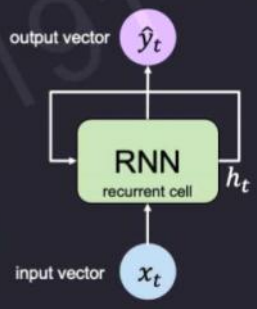
RNN Intuition

```
my_rnn = RNN()
hidden_state = [0, 0, 0, 0]

sentence = ["I", "love", "recurrent", "neural"]

for word in sentence:
    prediction, hidden_state = my_rnn(word, hidden_state)

next_word_prediction = prediction
# >>> "networks!"
```

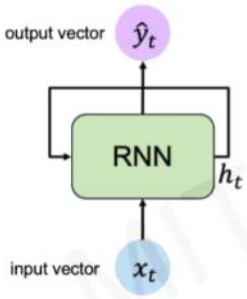


RNN Intuition

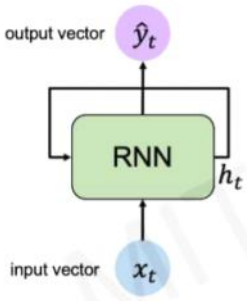
```
my_rnn = RNN()
hidden_state = [0, 0, 0, 0]

sentence = ["I", "love", "recurrent", "neural"]

for word in sentence:
    prediction, hidden_state = my_rnn(word, hidden_state)

next_word_prediction = prediction
# >>> "networks!"
```



RNN State Update and Output

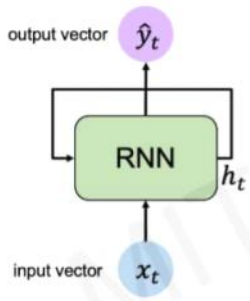


RNN State Update and Output



Input Vector
x_t

RNN State Update and Output



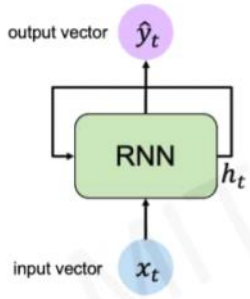
Update Hidden State

$$h_t = \tanh(\boldsymbol{W}_{hh}^T h_{t-1} + \boldsymbol{W}_{xh}^T x_t)$$

Input Vector

$$x_t$$

RNN State Update and Output



Output Vector

$$\hat{y}_t = \boldsymbol{W}_{hy}^T h_t$$

Two sets of weights for hidden state and input state

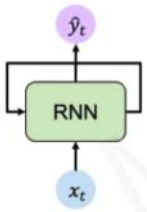
Update Hidden State

$$h_t = \tanh(\boldsymbol{W}_{hh}^T h_{t-1} + \boldsymbol{W}_{xh}^T x_t)$$

Input Vector

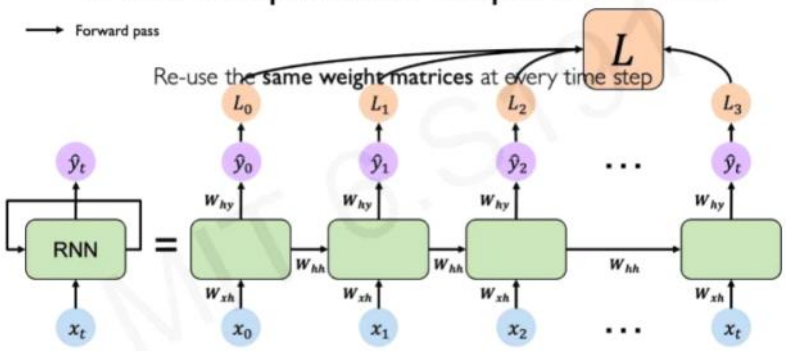
$$x_t$$

RNNs: Computational Graph Across Time



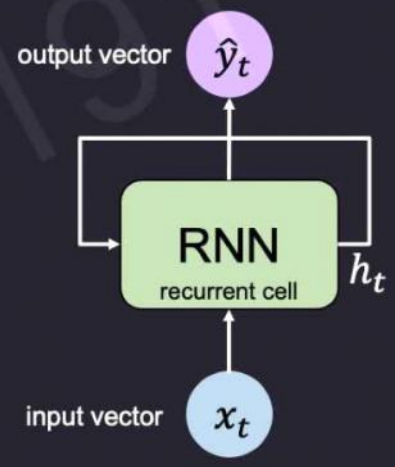
= Represent as computational graph unrolled across time

RNNs: Computational Graph Across Time



RNNs from Scratch in TensorFlow

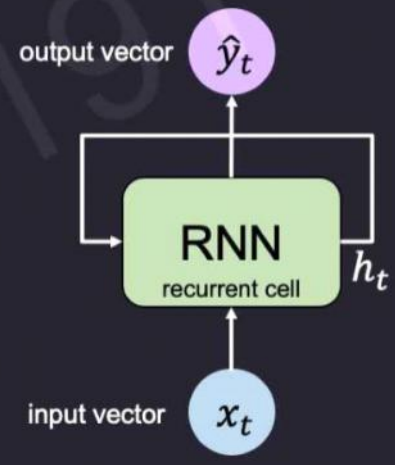
```
class MyRNNCell(tf.keras.layers.Layer):
  def __init__(self, rnn_units, input_dim, output_dim):
    super(MyRNNCell, self).__init__()

    # Initialize weight matrices
    self.W_xh = self.add_weight([rnn_units, input_dim])
    self.W_hh = self.add_weight([rnn_units, rnn_units])
    self.W_hy = self.add_weight([output_dim, rnn_units])

    # Initialize hidden state to zeros
    self.h = tf.zeros([rnn_units, 1])

  def call(self, x):
    # Update the hidden state
    self.h = tf.math.tanh( self.W_hh * self.h + self.W_xh * x )

    # Compute the output
    output = self.W_hy * self.h

    # Return the current output and hidden state
    return output, self.h
```



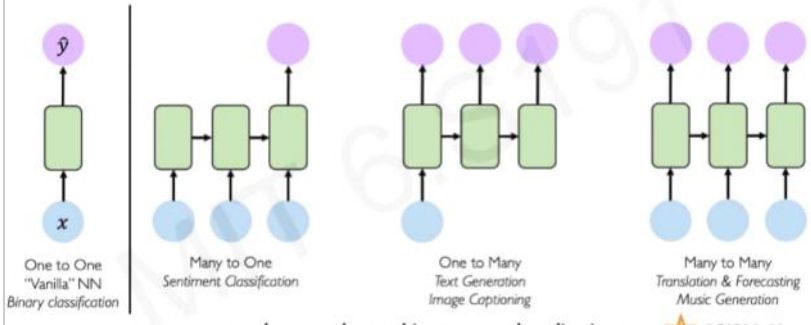
RNN Implementation: TensorFlow & PyTorch

```
from tf.keras.layers import SimpleRNN
model = SimpleRNN(rnn_units)
```

```
from torch.nn import RNN
model = RNN(input_size, rnn_units)
```



RNNs for Sequence Modeling



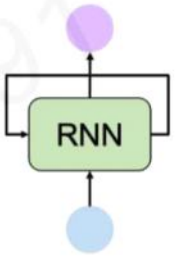
… and many other architectures and applications

6.S191 Lab!

Sequence Modeling: Design Criteria

To model sequences, we need to:

1. Handle **variable-length** sequences
2. Track **long-term** dependencies
3. Maintain information about **order**
4. **Share parameters** across the sequence



Recurrent Neural Networks (RNNs) meet these sequence modeling design criteria

A Sequence Modeling Problem: Predict the Next Word

A Sequence Modeling Problem: Predict the Next Word

"This morning I took my cat for a walk."

A Sequence Modeling Problem: Predict the Next Word

"This morning I took my cat for a walk."

given these words

A Sequence Modeling Problem: Predict the Next Word

"This morning I took my cat for a walk."

given these words

predict the next word

A Sequence Modeling Problem: Predict the Next Word

"This morning I took my cat for a walk."

given these words

predict the next word

Representing Language to a Neural Network

Neural networks cannot interpret words

Neural networks require numerical inputs

Encoding Language for a Neural Network

Neural networks cannot interpret words

Neural networks require numerical inputs

Embedding / Encoding - captures semantic content of the words in numerical form

Embedding: transform indexes into a vector of fixed size.

1. Vocabulary: Corpus of words

2. Indexing: Word to index

3. Embedding: Index to fixed-sized vector

NAÏVE Approach!

We need an embedding scheme to capture the semantics of words and relationships of words in the overall language

The embedding shouldn't just capture the word (in a one-hot encoding form) - it needs to capture the structure of language itself!

Handle Variable Sequence Lengths

The food was great

vs.

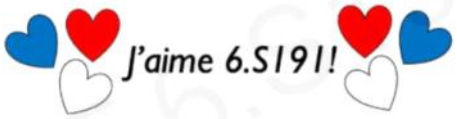
We visited a restaurant for lunch

vs.

We were hungry but cleaned the house before eating

Model Long-Term Dependencies

"**France** is where I grew up, but I now live in Boston. I speak fluent ___."



We need information from **the distant past** to accurately predict the correct word.

Capture Differences in Sequence Order

The food was good, not bad at all.

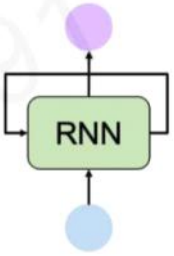
vs.

The food was bad, not good at all.

Sequence Modeling: Design Criteria

To model sequences, we need to:

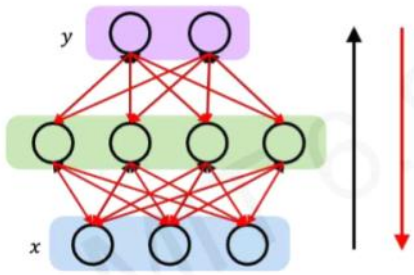
1. Handle **variable-length** sequences
2. Track **long-term** dependencies
3. Maintain information about **order**
4. **Share parameters** across the sequence



Recurrent Neural Networks (RNNs) meet these sequence modeling design criteria

Backpropagation Through Time (BPTT)

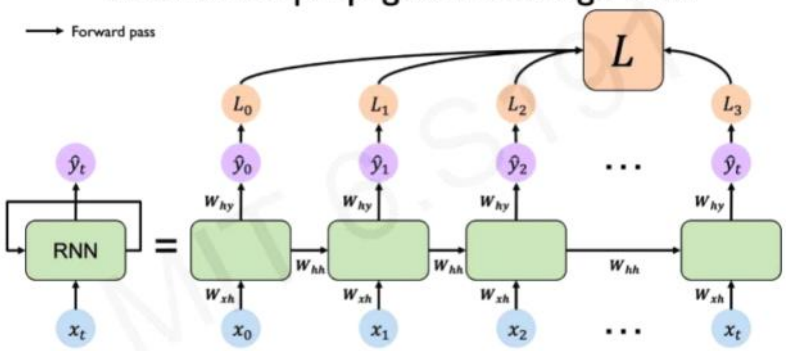
Recall: Backpropagation in Feed Forward Models



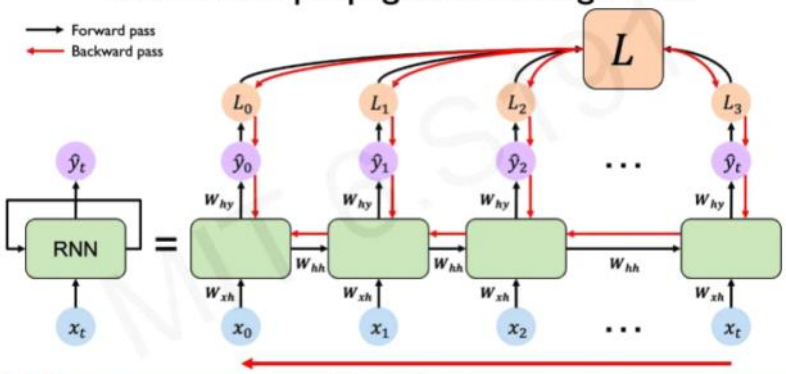
Backpropagation algorithm:

1. Take the derivative (gradient) of the loss with respect to each parameter
2. Shift parameters in order to minimize loss

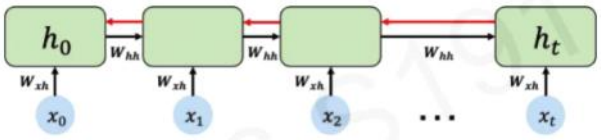
RNNs: Backpropagation Through Time



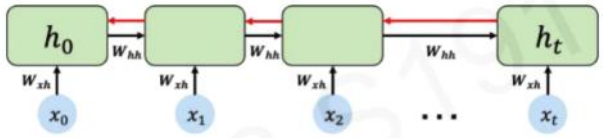
RNNs: Backpropagation Through Time



Standard RNN Gradient Flow

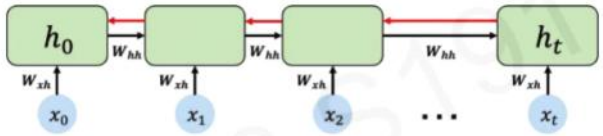


Standard RNN Gradient Flow



Computing the gradient wrt h_0 involves **many factors of W_{hh}** + **repeated gradient computation**!

Standard RNN Gradient Flow: Exploding Gradients

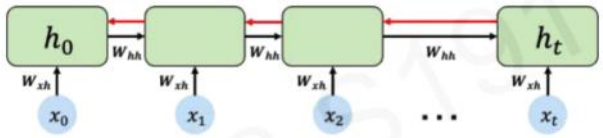


Computing the gradient wrt h_0 involves **many factors of W_{hh}** + **repeated gradient computation**!

Many values > 1:
exploding gradients

Gradient clipping to scale big gradients

Standard RNN Gradient Flow: Vanishing Gradients



Computing the gradient wrt h_0 involves **many factors of W_{hh}** + **repeated gradient computation**!

Many values < 1:
vanishing gradients

1. Activation function
2. Weight initialization
3. Network architecture

The Problem of Long-Term Dependencies

Why are vanishing gradients a problem?

Multiply many **small numbers** together

↓

Errors due to further back time steps have smaller and smaller gradients

The model becomes biased towards shorter-term inputs

↓

Bias parameters to capture short-term dependencies

The Problem of Long-Term Dependencies

Why are vanishing gradients a problem?

Multiply many **small numbers** together

↓

Errors due to further back time steps have smaller and smaller gradients

↓

Bias parameters to capture short-term dependencies

"The clouds are in the ___"

The Problem of Long-Term Dependencies

Why are vanishing gradients a problem?

Multiply many **small numbers** together

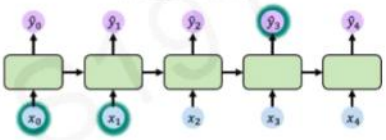
↓

Errors due to further back time steps have smaller and smaller gradients

↓

Bias parameters to capture short-term dependencies

"The clouds are in the ___"



The Problem of Long-Term Dependencies

Why are vanishing gradients a problem?

Multiply many **small numbers** together

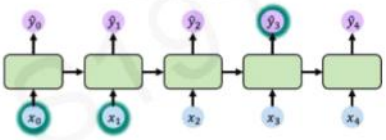
↓

Errors due to further back time steps have smaller and smaller gradients

↓

Bias parameters to capture short-term dependencies

"The clouds are in the ___"



"I grew up in France, … and I speak fluent___"

We lose information further back in time

The Problem of Long-Term Dependencies

Why are vanishing gradients a problem?

Multiply many **small numbers** together

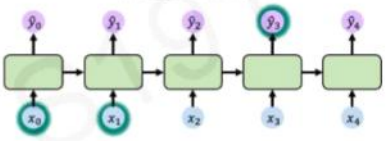
↓

Errors due to further back time steps have smaller and smaller gradients

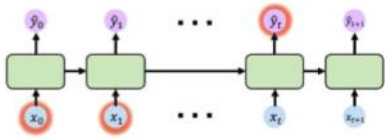
↓

Bias parameters to capture short-term dependencies

"The clouds are in the ___"

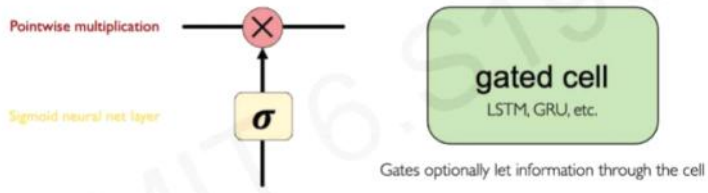


"I grew up in France, … and I speak fluent___"



Gating Mechanisms in Neurons

Idea: use **gates** to selectively **add** or **remove** information within **each recurrent unit with**

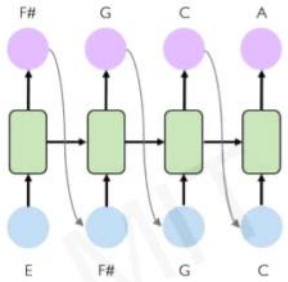


Gates optionally let information through the cell

Long Short Term Memory (LSTMs) networks rely on a gated cell to track information throughout many time steps.

RNN Applications & Limitations

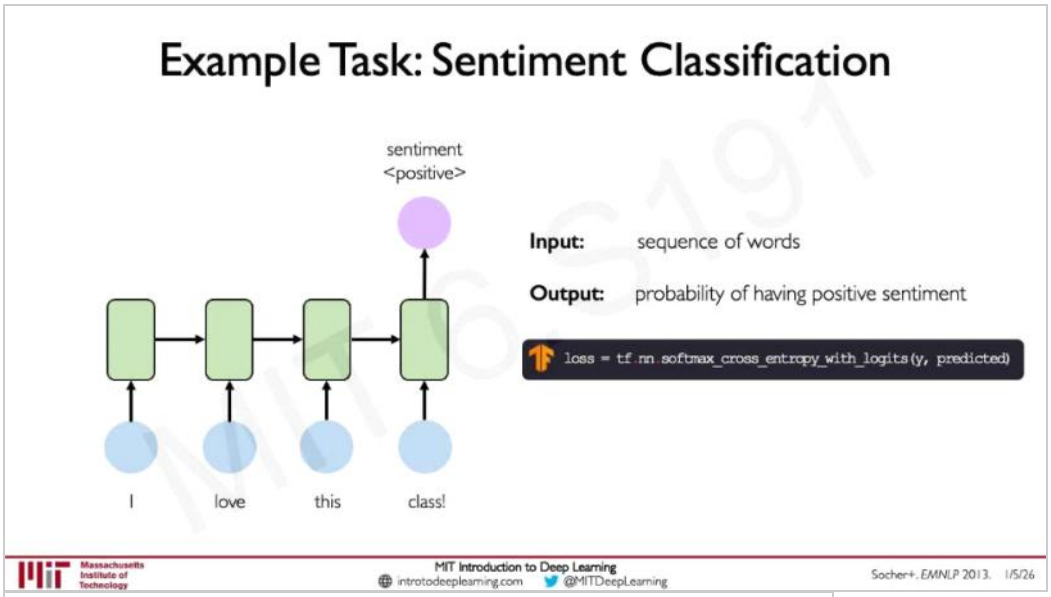
Example Task: Music Generation

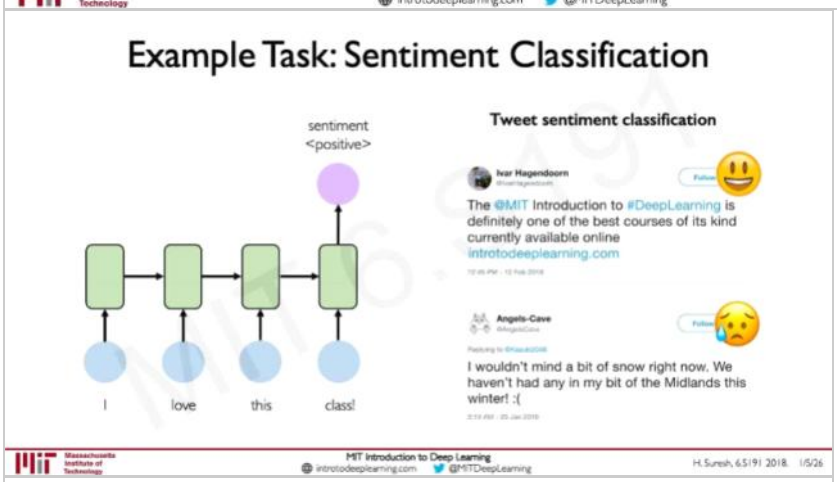


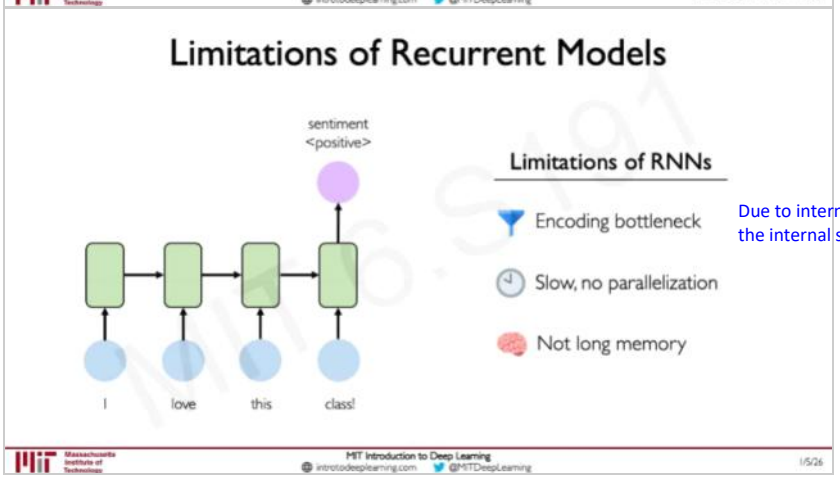
Input: sheet music

Output: next character in sheet music

6.S191 Lab!



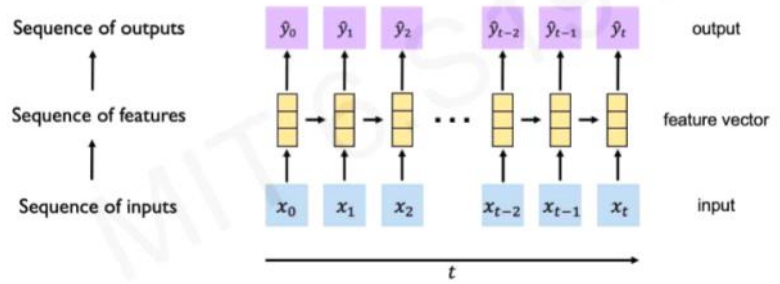




Due to internal state (all dependencies and structure is compacted into the internal state)

Goal of Sequence Modeling

RNNs: recurrence to model sequence dependencies

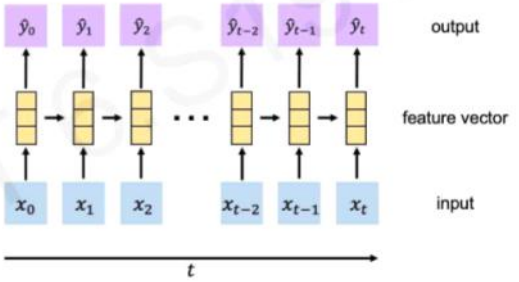


Goal of Sequence Modeling

RNNs: recurrence to model sequence dependencies

Limitations of RNNs

- Encoding bottleneck
- Slow, no parallelization
- Not long memory

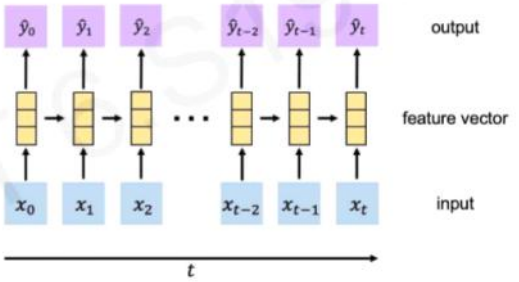


Goal of Sequence Modeling

Can we eliminate the need for recurrence entirely?

Desired Capabilities

- Continuous stream
- Parallelization
- Long memory

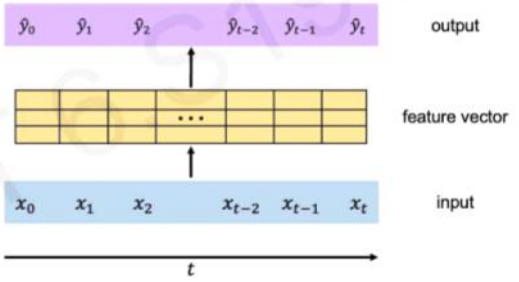


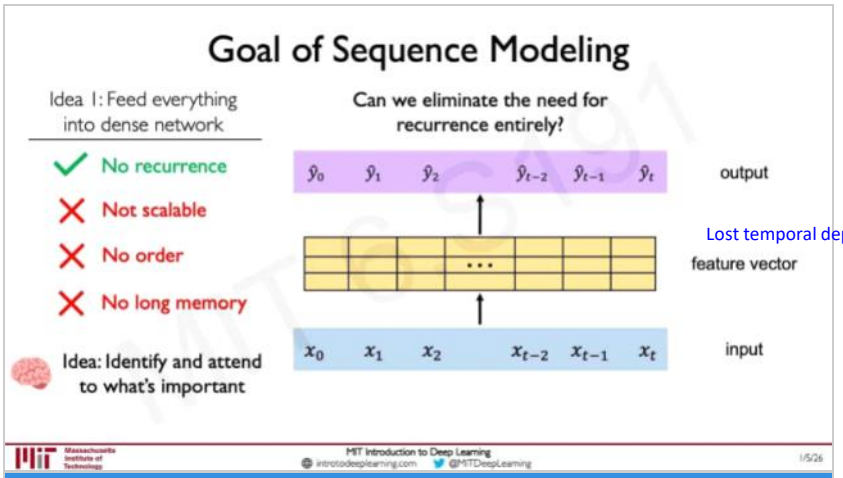
Goal of Sequence Modeling

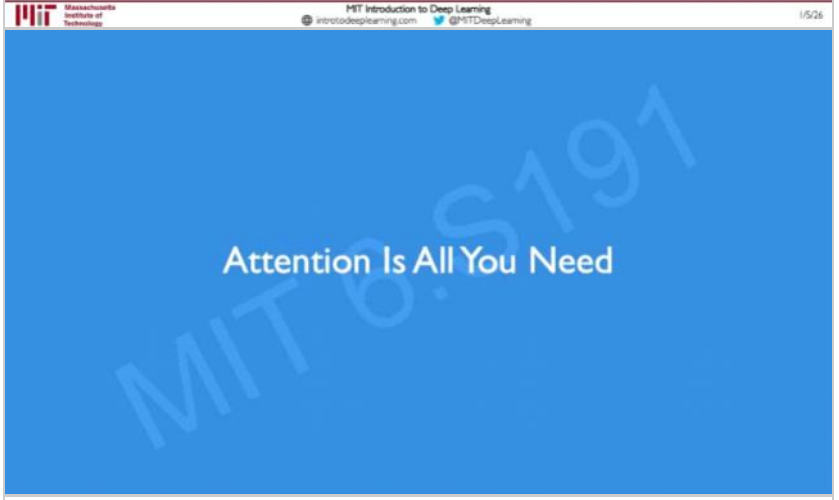
Can we eliminate the need for recurrence entirely?

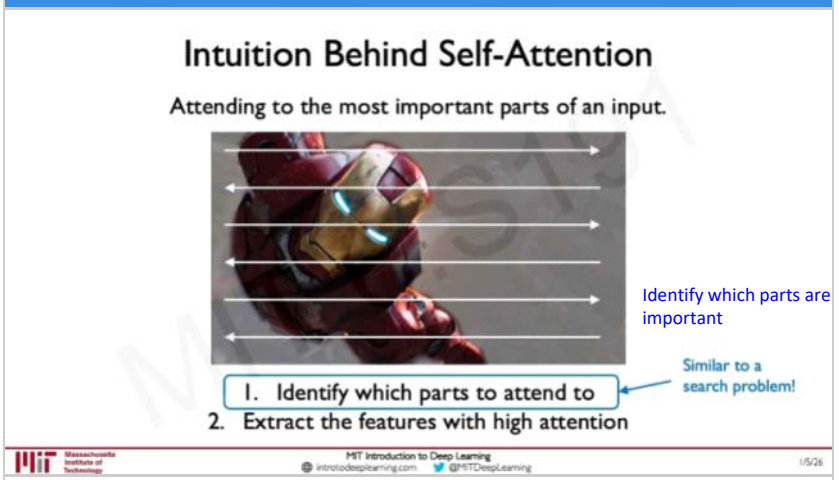
Desired Capabilities

- Continuous stream
- Parallelization
- x_0 Long memory

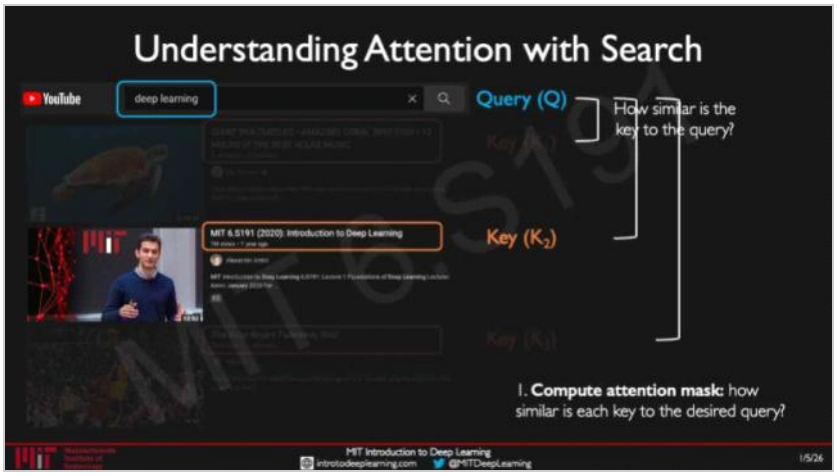


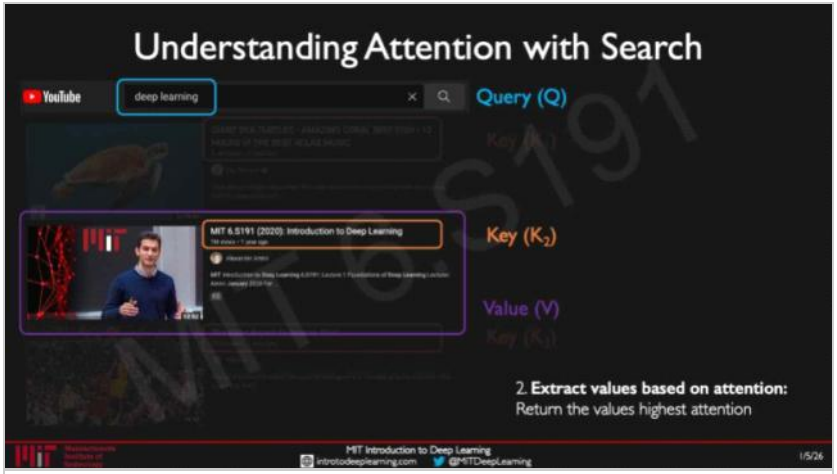

Goal of Sequence Modeling
Idea 1: Feed everything into dense network
No recurrence
Not scalable
No order
No long memory
Idea: Identify and attend to what's important
Can we eliminate the need for recurrence entirely?
$\hat{y}_0$ $\hat{y}_1$ $\hat{y}_2$ $\hat{y}_{t-2}$ $\hat{y}_{t-1}$ $\hat{y}_t$
output
Lost temporal dependency
feature vector
x_0 x_1 x_2 x_{t-2} x_{t-1} x_t
input
MIT Introduction to Deep Learning
introtodeeplearning.com
@MITDeepLearning
1/5/26

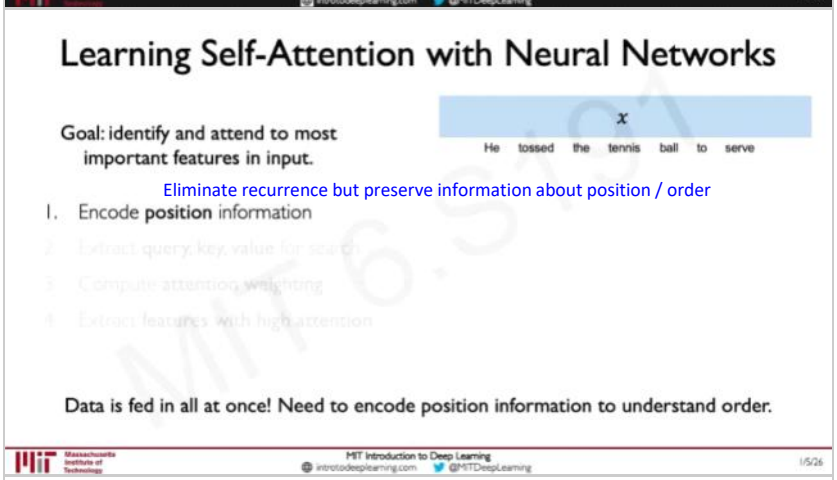

Attention Is All You Need

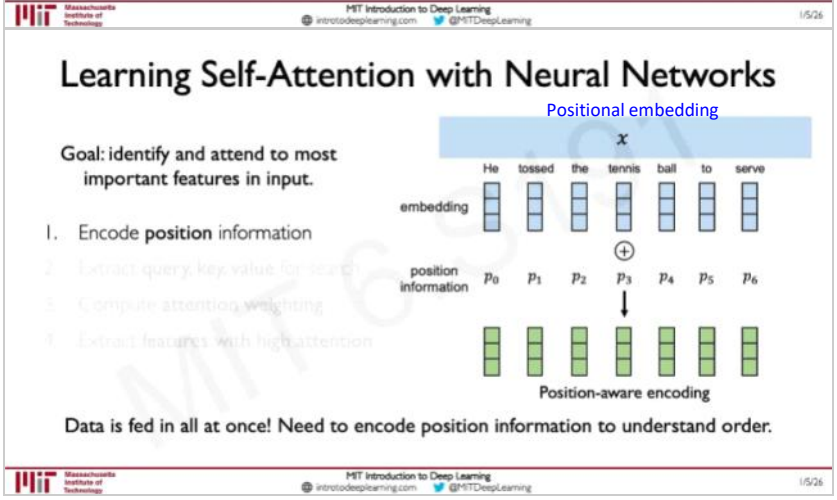

Intuition Behind Self-Attention
Attending to the most important parts of an input.
Identify which parts are important
Similar to a search problem!
1. Identify which parts to attend to
2. Extract the features with high attention
MIT Introduction to Deep Learning
introtodeeplearning.com
@MITDeepLearning
1/5/26

A Simple Example: Search
How can I learn more about neural networks?
MIT Introduction to Deep Learning
introtodeeplearning.com
@MITDeepLearning
1/5/26


Understanding Attention with Search
YouTube
deep learning
Query (Q)
How similar is the key to the query?
MIT 6.S191 (2020): Introduction to Deep Learning
Key (K_2)
1. Compute attention mask: how similar is each key to the desired query?
MIT Introduction to Deep Learning
introtodeeplearning.com
@MITDeepLearning
1/5/26


Understanding Attention with Search
YouTube
deep learning
Query (Q)
MIT 6.S191 (2020): Introduction to Deep Learning
Key (K_2)
Value (V)
2. Extract values based on attention: Return the values highest attention
MIT Introduction to Deep Learning
introtodeeplearning.com
@MITDeepLearning
1/5/26


Learning Self-Attention with Neural Networks
Goal: identify and attend to most important features in input.
x
He tossed the tennis ball to serve
Eliminate recurrence but preserve information about position / order
1. Encode position information
Data is fed in all at once! Need to encode position information to understand order.
MIT Introduction to Deep Learning
introtodeeplearning.com
@MITDeepLearning
1/5/26


Learning Self-Attention with Neural Networks
Positional embedding
Goal: identify and attend to most important features in input.
x
He tossed the tennis ball to serve
embedding
position information
p_0 p_1 p_2 p_3 p_4 p_5 p_6
1. Encode position information
Position-aware encoding
Data is fed in all at once! Need to encode position information to understand order.
MIT Introduction to Deep Learning
introtodeeplearning.com
@MITDeepLearning
1/5/26

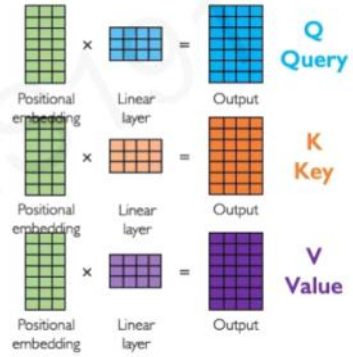
Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search

Positional embedding will be the same

But the learned layers for Q, K and V will be different because they capture different information



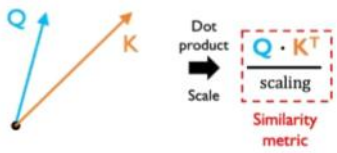
Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**

Attention score: compute pairwise similarity between each query and key

How to compute similarity between two sets of features?



Also known as the "cosine similarity"

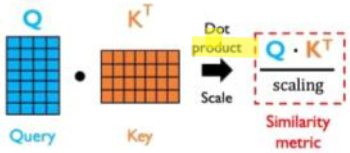
Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**

Attention score: compute pairwise similarity between each query and key

How to compute similarity between two sets of features?



Also known as the "cosine similarity"

Learning Self-Attention with Neural Networks

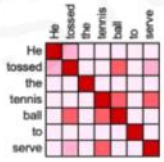
Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**

Diagonal has highest score - because words are same (Q and K are same)

But looking at the matrix, we will also be able to identify other words that are closely related to each other semantically

Attention weighting: where to attend to! How similar is the key to the query?



Softmax - values b/w 0 and 1

$$softmax\left(\frac{Q \cdot K^T}{scaling}\right)$$

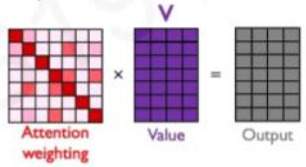
Attention weighting

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract **features with high attention**

Last step: self-attend to extract features



$$softmax\left(\frac{Q \cdot K^T}{scaling}\right) \cdot V = A(Q,K,V)$$

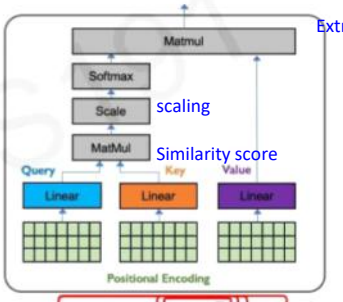
Attention weight

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract **features with high attention**

These operations form a self-attention head that can plug into a larger network. Each head attends to a different part of input.



Extract features with high attention

scaling

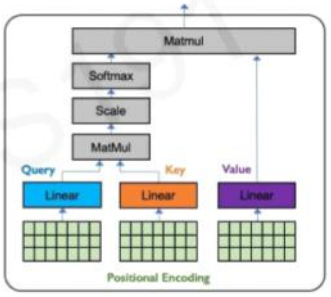
Similarity score

$$softmax\left(\frac{Q \cdot K^T}{scaling}\right) \cdot V$$

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract **features with high attention**



Attention is the foundational building block of the **Transformer** architecture.

Applying Multiple Self-Attention Heads

Attention weighting × Value = Output

Output of attention head 1 | Output of attention head 2 | Output of attention head 3

Self-Attention Applied

Language Processing

An armchair in the shape of an avocado

Transformers: BERT, GPT

Devlin et al., *NAACL* 2019
Brown et al., *NeurIPS* 2020

Biological Sequences

Protein Structure Models

Jumper et al., *Nature* 2021
Lin et al., *Science* 2023

Computer Vision

Vision Transformers

Dosovitskiy et al., *ICLR* 2020

6.S191 Lab and Lectures!

Deep Learning for Sequence Modeling: Summary

1. RNNs are well suited for **sequence modeling** tasks
2. Model sequences via a **recurrence relation**
3. Training RNNs with **backpropagation through time**
4. Models for **music generation**, classification, machine translation, and more
5. Self-attention to model **sequences without recurrence**
6. Self-attention is the basis for many **large language models** – stay tuned!

6.S191: Introduction to Deep Learning

Lab 1: Deep Learning in Python and Music Generation with RNNs

Link to download labs:
http://introtodeeplearning.com#schedule

1. Open the lab in Google Colab
2. Start executing code blocks and filling in the #TODOs
3. Need help? Find a TA/instructor!

6.S191: Introduction to Deep Learning

Kickoff Reception at One Kendall Square!

Register here: luma.com/47iswo1i

Join us at 5:00pm for a special kickoff reception at
1 Kendall Square!

Food and drinks will be provided!
Special thanks to John Werner and Link Ventures for hosting.